

Declaración responsable sobre autoría y uso ético de inteligencia artificial, otros recursos o ayuda externa

Con la entrega de esta tarea, declaro de manera responsable que es el resultado de mi trabajo intelectual personal y creativo, y que ha sido elaborado de acuerdo con los principios éticos y las normas de integridad vigentes en la comunidad académica y, más específicamente, en la [Universidad Complutense de Madrid](#).

Soy, pues, autor del material aquí incluido y, cuando no ha sido así y he tomado el material de otra fuente, lo he citado o bien he declarado su procedencia de forma clara -incluidas, en su caso, herramientas de inteligencia artificial-. Las ideas y aportaciones principales incluidas en este trabajo, y que acreditan la adquisición de competencias, son mías y no proceden de otras fuentes o han sido reescritas usando material de otras fuentes.

Antes de entregar esta tarea, asegúrate de que todo funcione como se espera:

1. Reinicia el kernel

(En la barra de menú, selecciona **Kernel** → **Restart**)

2. Ejecuta todas las celdas

(En la barra de menú, selecciona **Cell** → **Run All**)

3. Asegúrate de completar todos los lugares donde aparezca:

- YOUR CODE HERE
- "YOUR ANSWER HERE"

4. Debes borrar las instrucciones que pone `raise NotImplementedError()`

5. Así como tu **nombre** a continuación:

```
In [1]: NOMBRE = "Pablo Blas Hernández"
```

Notas para la entrega

- Se deben rellenar todos los cuadernos sin cambiar el nombre.
- Se debe comprimir la carpeta en un fichero `.zip`, llamado `tarea_final.zip`. **Importante:** por favor, no se entreguen ficheros `.rar`.
- Subir el fichero comprimido al campus virtual.

```
In [2]: # En esta casilla pon los import que necesites en este cuaderno.

import testing
import pandas as pd # Librería para manipulación de datos
import matplotlib.pyplot as plt # Librería de visualización de datos

DataFrame = pd.core.frame.DataFrame

# Las constantes que defines deben ir aquí detrás:
# Archivos de datos
DATAFILE = 'inscritos.csv'
TESTDATAFILE = 'inscritos_test_sample.csv'
```

Operaciones con dataframes

En este apartado, vamos a trabajar con tablas de la librería `pandas`, llamadas `dataframes`.

Carga del dataframe

La primera operación que necesitamos es cargar el archivo de datos en una tabla, como se ve en el siguiente ejemplo. Ten en cuenta que las columnas `FX_DATA` y `FECHA_INSCRIPCION` deben contener elementos de tipo fecha. La fecha en la columna `FX_DATA` no presenta mayor complicación, basta con indicarlo en el parámetro `parse_dates` de la función que lee el csv.

Pero como vimos, el paso a fecha de la columna `FECHA_INSCRIPCION` no funciona en algunos sistemas. Lo que vamos a hacer es tener dos columnas nuevas llamadas `MES_INSCRIPCION` y `ANYO_INSCRIPCION` (en ese orden) de forma que la primera contenga el mes como una cadena de caracteres y la segunda el día como un entero.

Un último detalle. Si visualizas el CSV (siguiente casilla), podrás observar que la descripción del distrito tiene un espacio adicional que hay que quitar.

```
In [3]: with open('inscritos.csv', encoding='utf-8') as fl:
        for _ in range(5):
            print(fl.readline().strip())
```

```
FECHA_INSCRIPCION;GENERO_DESC;DISTRITO_COD;DISTRITO_DESC;EDAD;NACIONALIDAD_DESC;OBJETIVOPROFESIONAL1_COD;OBJETIVOPROFESIONAL1_D
ESC;OBJETIVOPROFESIONAL2_COD;OBJETIVOPROFESIONAL2_DESC;OBJETIVOPROFESIONAL3_COD;OBJETIVOPROFESIONAL3_DESC;FX_CARGA
"ene-24";"Hombre";10;" LATINA";34;"Español";"5833";"Conserjes de edificios";"5220";"Vendedores en tiendas y almacenes";"960
2";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
"ene-20";"Hombre";10;" LATINA";40;"Extracomunitario";"5500";"Cajeros y taquilleros (excepto bancos);"5220";"Vendedores en tien
das y almacenes";"9602";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
"nov-23";"Hombre";13;" PUENTE DE VALLECAS";24;"Extracomunitario";"3510";"Agentes y representantes comerciales";"5420";"Operador
es de telemarketing";"9602";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
"oct-23";"Hombre";17;" VILLAVERDE";19;"Extracomunitario";"9820";"Reponedores";"5220";"Vendedores en tiendas y almacenes";"960
2";"Peones de la construcción de edificios";"2025-08-07 00:11:57.897"
```

```
In [4]: def load_dataframe(datafile: str) -> pd.DataFrame:
        """
        Carga un archivo csv (inscripciones de empleo) y realiza procesado básico:
        - Convierte la columna 'FX_CARGA' a tipo datetime
        - Crea las columnas 'MES_INSCRIPCION' y 'ANIO_INSCRIPCION' desde 'FECHA_INSCRIPCION'
        - Limpia espacios en blanco de la columna 'DISTRITO_DESC'
        - Reordena las columnas

        Args:
            datafile (str): Ruta del archivo csv con separador ';'

        Returns:
            pd.DataFrame: DataFrame procesado con las 15 columnas ordenadas

        Examples:
            >>> df = load_dataframe("inscritos.csv")
            >>> 'MES_INSCRIPCION' in df.columns
            True
            >>> 'ANYO_INSCRIPCION' in df.columns
            True
        """
```

```

# Convertimos FX_CARGA a tipo fecha
df = pd.read_csv(
    datafile,
    sep=';',
    parse_dates=['FX_CARGA']
)

# Obtenemos la ubicación de FECHA_INSCRIPCION para insertar las nuevas columnas
idx_fecha_inscripcion = df.columns.get_loc('FECHA_INSCRIPCION')

# Creamos las columnas MES_INSCRIPCION y ANYO_INSCRIPCION
df.insert(
    loc=idx_fecha_inscripcion + 1,
    column='MES_INSCRIPCION',
    value=df['FECHA_INSCRIPCION'].str[:3]
)

df.insert(
    loc=idx_fecha_inscripcion + 2,
    column='ANYO_INSCRIPCION',
    value=df['FECHA_INSCRIPCION'].str[-2:].astype(int)
)

# Limpiamos la columna DISTRITO_DESC
df['DISTRITO_DESC'] = df['DISTRITO_DESC'].str.strip()

# Creamos una lista con el orden deseado.
column_order = [
    'FECHA_INSCRIPCION', 'GENERO_DESC', 'DISTRITO_COD', 'DISTRITO_DESC', 'EDAD',
    'NACIONALIDAD_DESC', 'OBJETIVOPROFESIONAL1_COD', 'OBJETIVOPROFESIONAL1_DESC',
    'OBJETIVOPROFESIONAL2_COD', 'OBJETIVOPROFESIONAL2_DESC', 'OBJETIVOPROFESIONAL3_COD',
    'OBJETIVOPROFESIONAL3_DESC', 'FX_CARGA', 'MES_INSCRIPCION', 'ANYO_INSCRIPCION'
]

# Reordenamos el DataFrame antes de devolverlo
df = df[column_order]

return df

```

In [5]: *# Casilla para hacer pruebas*

```
df = load_dataframe(TESTDATAFILE)

print('MES_INSCRIPCION' in df.columns)  # Debe imprimir: True
print('ANYO_INSCRIPCION' in df.columns)  # Debe imprimir: True

print("-- Primeras 5 filas del DataFrame con columnas creadas y limpias --")
print(df.head())

print("\n-- Tipos de datos de las columnas (Verificar FX_CARGA y ANYO_INSCRIPCION) --")
df.info()
```

True

True

-- Primeras 5 filas del DataFrame con columnas creadas y limpias --

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD	\
0	jun-17	Hombre	22	OTRO MUNICIPIO	28.0	
1	may-23	Hombre	14	MORATALAZ	51.0	
2	jul-23	Hombre	13	PUENTE DE VALLECAS	49.0	
3	may-23	Mujer	12	USERA	41.0	
4	ene-23	Mujer	22	OTRO MUNICIPIO	43.0	

	NACIONALIDAD_DESC	OBJETIVOPROFESIONAL1_COD	\
0	Español	9229.0	
1	Español	3123.0	
2	Extracomunitario	9811.0	
3	Extracomunitario	5710.0	
4	Extracomunitario	9310.0	

	OBJETIVOPROFESIONAL1_DESC	\
0	Otro personal de limpieza	
1	Técnicos en electricidad	
2	Peones del transporte de mercancías y descarga...	
3	Trabajadores de los cuidados personales a domi...	
4	Ayudantes de cocina	

	OBJETIVOPROFESIONAL2_COD	\
0	9820.0	
1	7510.0	
2	7240.0	
3	9210.0	
4	7835.0	

	OBJETIVOPROFESIONAL2_DESC	\
0	Reponedores	
1	Electricistas de la construcción y afines	
2	Soladores, colocadores de parquet y afines	
3	Personal de limpieza de oficinas, hoteles y ot...	
4	Tapiceros, colchoneros y afines	

	OBJETIVOPROFESIONAL3_COD	OBJETIVOPROFESIONAL3_DESC	\
0	5220.0	Vendedores en tiendas y almacenes	
1	9602.0	Peones de la construcción de edificios	

```

2          9602.0   Peones de la construcción de edificios
3          9700.0   Peones de las industrias manufactureras
4          9700.0   Peones de las industrias manufactureras

```

```

          FX_CARGA MES_INSCRIPCION  ANYO_INSCRIPCION
0 2025-08-07 00:11:57.897          jun             17
1 2025-08-07 00:11:57.897          may             23
2 2025-08-07 00:11:57.897          jul             23
3 2025-08-07 00:11:57.897          may             23
4 2025-08-07 00:11:57.897          ene             23

```

```
-- Tipos de datos de las columnas (Verificar FX_CARGA y ANYO_INSCRIPCION) --
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1508 entries, 0 to 1507
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	FECHA_INSCRIPCION	1508 non-null	object
1	GENERO_DESC	1508 non-null	object
2	DISTRITO_COD	1508 non-null	int64
3	DISTRITO_DESC	1508 non-null	object
4	EDAD	1505 non-null	float64
5	NACIONALIDAD_DESC	1508 non-null	object
6	OBJETIVOPROFESIONAL1_COD	1385 non-null	float64
7	OBJETIVOPROFESIONAL1_DESC	1385 non-null	object
8	OBJETIVOPROFESIONAL2_COD	1148 non-null	float64
9	OBJETIVOPROFESIONAL2_DESC	1148 non-null	object
10	OBJETIVOPROFESIONAL3_COD	949 non-null	float64
11	OBJETIVOPROFESIONAL3_DESC	949 non-null	object
12	FX_CARGA	1508 non-null	datetime64[ns]
13	MES_INSCRIPCION	1508 non-null	object
14	ANYO_INSCRIPCION	1508 non-null	int64

```
dtypes: datetime64[ns](1), float64(4), int64(2), object(8)
```

```
memory usage: 176.8+ KB
```

Auto-comprobación

In [6]: *# Prueba:*

```
full_data = load_dataframe("inscritos.csv")
```

```
print(full_data.dtypes)
full_data
```

FECHA_INSCRIPCION	object
GENERO_DESC	object
DISTRITO_COD	int64
DISTRITO_DESC	object
EDAD	int64
NACIONALIDAD_DESC	object
OBJETIVOPROFESIONAL1_COD	float64
OBJETIVOPROFESIONAL1_DESC	object
OBJETIVOPROFESIONAL2_COD	float64
OBJETIVOPROFESIONAL2_DESC	object
OBJETIVOPROFESIONAL3_COD	float64
OBJETIVOPROFESIONAL3_DESC	object
FX_CARGA	datetime64[ns]
MES_INSCRIPCION	object
ANYO_INSCRIPCION	int64

dtype: object

Out[6]:

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD	NACIONALIDAD_DESC	OBJETIVOPROFESIONAL1_COD
0	ene-24	Hombre	10	LATINA	34	Español	5833.C
1	ene-20	Hombre	10	LATINA	40	Extracomunitario	5500.C
2	nov-23	Hombre	13	PUENTE DE VALLECAS	24	Extracomunitario	3510.C
3	oct-23	Hombre	17	VILLAVERDE	19	Extracomunitario	9820.C
4	may-23	Hombre	17	VILLAVERDE	42	Extracomunitario	9512.C
...	...	...	...	...	...	...	..
150132	may-23	Mujer	11	CARABANCHEL	33	Extracomunitario	5500.C
150133	mar-19	Mujer	22	OTRO MUNICIPIO	50	Extracomunitario	8193.C
150134	may-21	Hombre	17	VILLAVERDE	35	Extracomunitario	8333.C
150135	dic-18	Hombre	2	ARGANZUELA	42	Español	9811.C
150136	feb-17	Hombre	11	CARABANCHEL	48	Extracomunitario	4422.C

150137 rows × 15 columns



Salida esperada



```

FECHA_INSCRIPCION      object
GENERO_DESC            object
DISTRITO_COD           int64
DISTRITO_DESC          object
EDAD                  int64
NACIONALIDAD_DESC      object
OBJETIVOPROFESIONAL1_COD float64
OBJETIVOPROFESIONAL1_DESC object
OBJETIVOPROFESIONAL2_COD float64
OBJETIVOPROFESIONAL2_DESC object
OBJETIVOPROFESIONAL3_COD float64
OBJETIVOPROFESIONAL3_DESC object
FX_CARGA               datetime64[ns]
MES_INSCRIPCION        object
ANYO_INSCRIPCION       int64
dtype: object

```

6]:

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD	NACIONALIDAD_DESC	OBJETIVOPROFESIONAL1_COD	OBJETT
0	ene-24	Hombre	10	LATINA	34	Español	5833.0	
1	ene-20	Hombre	10	LATINA	40	Extracomunitario	5500.0	Caje
2	nov-23	Hombre	13	PUENTE DE VALLECAS	24	Extracomunitario	3510.0	
3	oct-23	Hombre	17	VILLABERDE	19	Extracomunitario	9820.0	
4	may-23	Hombre	17	VILLABERDE	42	Extracomunitario	9512.0	Peo
...	...	...	...	...	...	...	...	
150132	may-23	Mujer	11	CARABANCHEL	33	Extracomunitario	5500.0	Caje
150133	mar-19	Mujer	22	OTRO MUNICIPIO	50	Extracomunitario	8193.0	Op
150134	may-21	Hombre	17	VILLABERDE	35	Extracomunitario	8333.0	
150135	dic-18	Hombre	2	ARGANZUELA	42	Español	9811.0	

150136

feb-17

Hombre

11

CARABANCHEL

48

Extracomunitario

4422.0

F

150137 rows x 15 columns

Comprobaciones automáticas

```
In [7]: testing.test_load_dataframe(load_dataframe)
```

```
size = (150137, 15).....OK
El tipo de la columna FECHA_INSCRIPCION es object....OK
El tipo de la columna GENERO_DESC es object....OK
El tipo de la columna DISTRITO_COD es int64....OK
El tipo de la columna DISTRITO_DESC es object....OK
El tipo de la columna EDAD es int64....OK
El tipo de la columna NACIONALIDAD_DESC es object....OK
El tipo de la columna OBJETIVOPROFESIONAL1_COD es float64....OK
El tipo de la columna OBJETIVOPROFESIONAL1_DESC es object....OK
El tipo de la columna OBJETIVOPROFESIONAL2_COD es float64....OK
El tipo de la columna OBJETIVOPROFESIONAL2_DESC es object....OK
El tipo de la columna OBJETIVOPROFESIONAL3_COD es float64....OK
El tipo de la columna OBJETIVOPROFESIONAL3_DESC es object....OK
El tipo de la columna FX_CARGA es datetime64[ns]....OK
El tipo de la columna MES_INSCRIPCION es object....OK
El tipo de la columna ANYO_INSCRIPCION es int64....OK
OK
```

```
In [8]: testing.test_type_hints_load_dataframe(load_dataframe)
```

```
load_dataframe type hints OK
```

```
In [9]: testing.test_docstring(load_dataframe)
```

```
docstring OK
```

```
In [10]: testing.test_doctests(load_dataframe, globals())
```

```
doctests OK
```

Una tabla más reducida

En efecto: no vamos a necesitar las columnas de los objetivos profesionales. Define una función `reduce_cols` que devuelva una tabla reducida con las columnas deseadas.

```
In [11]: def reduce_cols(df: DataFrame) -> DataFrame:
        """
        Elimina las columnas relacionadas con objetivos profesionales para simplificar el análisis.
        Reduce el Dataframe de 15 a 9 columnas eliminando todas aquellas que contienen 'OBJETIVOPROFESIONAL'

        Args:
            df (DataFrame): DataFrame con 15 columnas (resultado de load_dataframe)

        Returns:
            DataFrame: DataFrame simplificado con 9 columnas esenciales

        Examples:
            >>> df_full = load_dataframe("inscritos.csv")
            >>> df_mini = reduce_cols(df_full)
            >>> len(df_mini.columns) < len(df_full.columns)
            True
        """
        # Eliminamos todas las columnas que contienen el texto 'OBJETIVOPROFESIONAL'
        cols_to_drop = [
            col for col in df.columns
            if 'OBJETIVOPROFESIONAL' in col
        ]

        df_reduced = df.drop(columns=cols_to_drop)

        return df_reduced
```

```
In [12]: # Casilla para hacer pruebas

df_full = load_dataframe(TESTDATAFILE)
print(f"Columnas originales (esperado 15): {len(df_full.columns)}")

df_mini = reduce_cols(df_full)
```

```
print(f"Columnas después de la reducción: {len(df_mini.columns)}")
print(len(df_mini.columns) < len(df_full.columns))    # Debe imprimir: True

# Verificaciones
# a) Confirmar el número de columnas (Debería pasar de 15 a 9)
expected_cols = 9
if len(df_mini.columns) == expected_cols:
    print(f"Número de columnas correcto ({expected_cols}).")
else:
    print(f"Error: Número de columnas. Esperado: {expected_cols}, Obtenido: {len(df_mini.columns)}.")

# b) Mostrar Las columnas resultantes y verificar que no quedan columnas de objetivos
objetivo_cols_present = [col for col in df_mini.columns if 'OBJETIVOPROFESIONAL' in col]

if not objetivo_cols_present:
    print("Se han eliminado todas las columnas 'OBJETIVOPROFESIONAL'.")
    print("Columnas restantes:")
    print(df_mini.columns.tolist())
else:
    print(f"Error: Columnas 'OBJETIVOPROFESIONAL' encontradas: {objetivo_cols_present}")

# c) Mostrar Las primeras filas del DataFrame reducido
print("\nPrimeras 5 filas del DataFrame reducido:")
print(df_mini.head())
```

Columnas originales (esperado 15): 15

Columnas después de la reducción: 9

True

Número de columnas correcto (9).

Se han eliminado todas las columnas 'OBJETIVOPROFESIONAL'.

Columnas restantes:

```
['FECHA_INSCRIPCION', 'GENERO_DESC', 'DISTRITO_COD', 'DISTRITO_DESC', 'EDAD', 'NACIONALIDAD_DESC', 'FX_CARGA', 'MES_INSCRIPCION', 'ANYO_INSCRIPCION']
```

Primeras 5 filas del DataFrame reducido:

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD
0	jun-17	Hombre	22	OTRO MUNICIPIO	28.0
1	may-23	Hombre	14	MORATALAZ	51.0
2	jul-23	Hombre	13	PUENTE DE VALLECAS	49.0
3	may-23	Mujer	12	USERA	41.0
4	ene-23	Mujer	22	OTRO MUNICIPIO	43.0

	NACIONALIDAD_DESC	FX_CARGA	MES_INSCRIPCION	ANYO_INSCRIPCION
0	Español	2025-08-07 00:11:57.897	jun	17
1	Español	2025-08-07 00:11:57.897	may	23
2	Extracomunitario	2025-08-07 00:11:57.897	jul	23
3	Extracomunitario	2025-08-07 00:11:57.897	may	23
4	Extracomunitario	2025-08-07 00:11:57.897	ene	23

Auto-comprobacion

```
In [13]: mini = reduce_cols(full_data)
print(mini.dtypes)
mini
```

```
FECHA_INSCRIPCION    object
GENERO_DESC          object
DISTRITO_COD         int64
DISTRITO_DESC        object
EDAD                int64
NACIONALIDAD_DESC    object
FX_CARGA             datetime64[ns]
MES_INSCRIPCION      object
ANYO_INSCRIPCION     int64
dtype: object
```

Out[13]:

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD	NACIONALIDAD_DESC	FX_CARGA	MES_INSCRIPCION
0	ene-24	Hombre	10	LATINA	34	Español	2025-08-07 00:11:57.897	e
1	ene-20	Hombre	10	LATINA	40	Extracomunitario	2025-08-07 00:11:57.897	e
2	nov-23	Hombre	13	PUENTE DE VALLECAS	24	Extracomunitario	2025-08-07 00:11:57.897	n
3	oct-23	Hombre	17	VILLAVERDE	19	Extracomunitario	2025-08-07 00:11:57.897	c
4	may-23	Hombre	17	VILLAVERDE	42	Extracomunitario	2025-08-07 00:11:57.897	m
...	...	...	...	...	...	...	...	...
150132	may-23	Mujer	11	CARABANCHEL	33	Extracomunitario	2025-08-07 00:12:50.377	m
150133	mar-19	Mujer	22	OTRO MUNICIPIO	50	Extracomunitario	2025-08-07 00:12:50.377	m
150134	may-21	Hombre	17	VILLAVERDE	35	Extracomunitario	2025-08-07 00:12:50.377	m
150135	dic-18	Hombre	2	ARGANZUELA	42	Español	2025-08-07 00:12:50.377	c
150136	feb-17	Hombre	11	CARABANCHEL	48	Extracomunitario	2025-08-07 00:12:50.377	f

150137 rows × 9 columns



Salida esperada

```
FECHA_INSCRIPCION      object
GENERO_DESC             object
DISTRITO_COD           int64
DISTRITO_DESC          object
EDAD                   int64
NACIONALIDAD_DESC      object
FX_CARGA               datetime64[ns]
MES_INSCRIPCION         object
ANYO_INSCRIPCION        int64
dtype: object
```

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD	NACIONALIDAD_DESC	FX_CARGA	MES_INSCRIPCION	ANYC
0	ene-24	Hombre	10	LATINA	34	Español	2025-08-07 00:11:57.897	ene	
1	ene-20	Hombre	10	LATINA	40	Extracomunitario	2025-08-07 00:11:57.897	ene	
2	nov-23	Hombre	13	PUENTE DE VALLECAS	24	Extracomunitario	2025-08-07 00:11:57.897	nov	
3	oct-23	Hombre	17	VILLAVERDE	19	Extracomunitario	2025-08-07 00:11:57.897	oct	
4	may-23	Hombre	17	VILLAVERDE	42	Extracomunitario	2025-08-07 00:11:57.897	may	
...	...	...	...	...	...	...	...	...	...
150132	may-23	Mujer	11	CARABANCHEL	33	Extracomunitario	2025-08-07 00:12:50.377	may	
150133	mar-19	Mujer	22	OTRO MUNICIPIO	50	Extracomunitario	2025-08-07 00:12:50.377	mar	
150134	may-21	Hombre	17	VILLAVERDE	35	Extracomunitario	2025-08-07 00:12:50.377	may	
150135	dic-18	Hombre	2	ARGANZUELA	42	Español	2025-08-07 00:12:50.377	dic	
150136	feb-17	Hombre	11	CARABANCHEL	48	Extracomunitario	2025-08-07 00:12:50.377	feb	

150137 rows × 9 columns

Comprobaciones automáticas

```
In [14]: testing.test_reduce_cols(reduce_cols, load_dataframe)
```

```
size = (150137, 9).....OK
Columnas..... OK
El tipo de la columna FECHA_INSCRIPCION es object....OK
El tipo de la columna GENERO_DESC es object....OK
El tipo de la columna DISTRITO_COD es int64....OK
El tipo de la columna DISTRITO_DESC es object....OK
El tipo de la columna EDAD es int64....OK
El tipo de la columna NACIONALIDAD_DESC es object....OK
El tipo de la columna FX_CARGA es datetime64[ns]....OK
El tipo de la columna MES_INSCRIPCION es object....OK
El tipo de la columna ANYO_INSCRIPCION es int64....OK
```

```
In [15]: testing.test_type_hints_reduce_cols(reduce_cols)
```

```
reduce_cols type hints OK
reduce_cols type hints OK
```

```
In [16]: testing.test_docstring(reduce_cols)
```

```
docstring OK
```

```
In [17]: testing.test_doctests(reduce_cols, globals())
```

```
doctests OK
```

Selección de un año

Define una función `filter_year` que dado un DataFrame nos devuelva las columnas que se correspondan con un determinado año.

```
In [18]: import pandas as pd
         from pandas import DataFrame

         def filter_year(df: DataFrame, year: int) -> DataFrame:
             """
```

Filtra el DataFrame para obtener únicamente las inscripciones de un año específico.

La función convierte automáticamente el año de 4 dígitos (e.g. 2024) a dos dígitos (24). Esto se compara con el año de la columna 'ANYO_INSCRIPCION' que está en formato corto.

Args:

df (DataFrame): DataFrame con la columna ANYO_INSCRIPCION en formato de dos dígitos
year (int): Año completo a filtrar (e.g. 2022, 2024)

Returns:

DataFrame: Subconjunto del DataFrame con solo a las filas del año solicitado

Examples:

```
# Se requiere definir df_mini o simularlo en el entorno de prueba
# Asumiendo que 'df_mini' existe y está cargado
# Si filtramos por 2024, el resultado debe tener el ANYO_INSCRIPCION = 24
>>> df_full = load_dataframe("inscritos_test_sample.csv")
>>> df_mini = reduce_cols(df_full)
>>> df_2024 = filter_year(df_mini, 2024)
>>> df_2024['ANYO_INSCRIPCION'].unique().tolist()
[24]
"""

# Obtenemos el año en formato de dos dígitos (e.g. 2022 % 100 = 22)
# Se usa el módulo 100 para obtener los dos últimos dígitos.
year_two_digits = year % 100

df_filtered = df[df['ANYO_INSCRIPCION'] == year_two_digits]

return df_filtered
```

In [19]: *# Casilla para hacer pruebas*

```
df_full = load_dataframe(TESTDATAFILE)
df_mini = reduce_cols(df_full)
print(f"Número total de filas en df_mini: {len(df_mini)}")
print(f"Años únicos presentes en el DataFrame: {df_mini['ANYO_INSCRIPCION'].unique().tolist()}")

df_2024 = filter_year(df_mini, 2024)
print(df_2024['ANYO_INSCRIPCION'].unique().tolist())    # Debe imprimir: [24]
```

```
# Definimos el año a filtrar y el valor esperado de dos dígitos
TEST_YEAR = 2022
EXPECTED_YEAR_DIGITS = TEST_YEAR % 100 # Debería ser 22
print(f"\n-- Filtrando por el año: {TEST_YEAR} (Dígitos esperados: {EXPECTED_YEAR_DIGITS}) --")

df_filtered = filter_year(df_mini, TEST_YEAR)

# Verificaciones
# a) Verificar que el DataFrame filtrado no esté vacío
if not df_filtered.empty:
    print("El DataFrame filtrado no está vacío.")
else:
    print(f"Error: El DataFrame está vacío. No se encontraron datos para el año {TEST_YEAR}.")

# b) Verificar que solo contiene el año esperado
unique_years = df_filtered['ANYO_INSCRIPCION'].unique().tolist()

if len(unique_years) == 1 and unique_years[0] == EXPECTED_YEAR_DIGITS:
    print(f"Solo se encontró el año {EXPECTED_YEAR_DIGITS} en los resultados.")
else:
    print(f"Error: El filtro devolvió años inesperados: {unique_years}. Se esperaba solo [{EXPECTED_YEAR_DIGITS}].")

# c) Mostrar la forma y las primeras filas del resultado
print(f"\nForma del DataFrame filtrado (Filas, Columnas): {df_filtered.shape}")
print("Primeras 5 filas del DataFrame filtrado:")
print(df_filtered[['FECHA_INSCRIPCION', 'ANYO_INSCRIPCION', 'EDAD']].head())
```

```
Número total de filas en df_mini: 1508
Años únicos presentes en el DataFrame: [17, 23, 20, 22, 19, 24, 25, 21, 18]
[24]
```

```
-- Filtrando por el año: 2022 (Dígitos esperados: 22) --
El DataFrame filtrado no está vacío.
Solo se encontró el año 22 en los resultados.
```

```
Forma del DataFrame filtrado (Filas, Columnas): (217, 9)
```

```
Primeras 5 filas del DataFrame filtrado:
```

	FECHA_INSCRIPCION	ANYO_INSCRIPCION	EDAD
6	sep-22	22	21.0
16	feb-22	22	33.0
18	ene-22	22	32.0
32	oct-22	22	21.0
36	jul-22	22	21.0

Auto-comprobación

```
In [20]: year_2022 = filter_year(mini, 2022)
year_2022
```

Out[20]:

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD	NACIONALIDAD_DESC	FX_CARGA	MES_INSCRIPCION
9	ago-22	Hombre	3	RETIRO	57	Extracomunitario	2025-08-07 00:11:57.897	ago-22
14	jun-22	Hombre	22	OTRO MUNICIPIO	21	Extracomunitario	2025-08-07 00:11:57.897	jun-22
18	feb-22	Hombre	13	PUENTE DE VALLECAS	47	Español	2025-08-07 00:11:57.897	feb-22
22	ago-22	Mujer	11	CARABANCHEL	27	Español	2025-08-07 00:11:57.897	ago-22
27	jul-22	Hombre	6	TETUÁN	35	Extracomunitario	2025-08-07 00:11:57.897	jul-22
...	...	...	...	...	...	...	...	...
150072	dic-22	Mujer	20	SAN BLAS - CANILLEJAS	38	Español	2025-08-07 00:12:50.377	dic-22
150090	sep-22	Hombre	21	BARAJAS	20	Extracomunitario	2025-08-07 00:12:50.377	sep-22
150113	abr-22	Hombre	13	PUENTE DE VALLECAS	25	Extracomunitario	2025-08-07 00:12:50.377	abr-22
150121	mar-22	Hombre	10	LATINA	25	Comunitario	2025-08-07 00:12:50.377	mar-22
150130	mar-22	Hombre	18	VILLA DE VALLECAS	33	Español	2025-08-07 00:12:50.377	mar-22

21199 rows × 9 columns



Salida esperada

	FECHA_INSCRIPCION	GENERO_DESC	DISTRITO_COD	DISTRITO_DESC	EDAD	NACIONALIDAD_DESC	FX_CARGA	MES_INSCRIPCION	ANYC
9	ago-22	Hombre	3	RETIRO	57	Extracomunitario	2025-08-07 00:11:57.897	ago	
14	jun-22	Hombre	22	OTRO MUNICIPIO	21	Extracomunitario	2025-08-07 00:11:57.897	jun	
18	feb-22	Hombre	13	PUENTE DE VALLECAS	47	Español	2025-08-07 00:11:57.897	feb	
22	ago-22	Mujer	11	CARABANCHEL	27	Español	2025-08-07 00:11:57.897	ago	
27	jul-22	Hombre	6	TETUÁN	35	Extracomunitario	2025-08-07 00:11:57.897	jul	
...	...	...	...	...	...	...	...	...	...
150072	dic-22	Mujer	20	SAN BLAS - CANILLEJAS	38	Español	2025-08-07 00:12:50.377	dic	
150090	sep-22	Hombre	21	BARAJAS	20	Extracomunitario	2025-08-07 00:12:50.377	sep	
150113	abr-22	Hombre	13	PUENTE DE VALLECAS	25	Extracomunitario	2025-08-07 00:12:50.377	abr	
150121	mar-22	Hombre	10	LATINA	25	Comunitario	2025-08-07 00:12:50.377	mar	
150130	mar-22	Hombre	18	VILLA DE VALLECAS	33	Español	2025-08-07 00:12:50.377	mar	

21199 rows × 9 columns

Comprobaciones automáticas

```
In [21]: testing.test_filter_year(filter_year, load_dataframe)
```

OK

```
In [22]: testing.test_type_hints_filter_year(filter_year)
```

```
filter_year type hints OK
filter_year type hints OK
```

```
In [23]: testing.test_docstring(filter_year)
```

```
docstring OK
```

```
In [24]: testing.test_doctests(filter_year, globals())
```

```
doctests OK
```

Tanteo por meses

Realiza una función `by_month` que dado el DataFrame de un año, obtenga una lista ordenada de tuplas de la forma `(mes, total)` tal que el total es el número de personas que se han inscrito en ese mes. La función deberá hacer un agrupamiento en el DataFrame y transformar el resultado para que finalmente devuelva algo del tipo `list[tuple[str, int]]`. La lista deberá estar ordenada con respecto al mes.

```
In [25]: def by_month(df: DataFrame) -> 'list[tuple[str, int]]':
        """
```

```
    Calcula el número de inscripciones por mes en orden cronológico.
```

```
    Agrupa por MES_INSCRIPCION y cuenta inscripciones.
```

```
    Usa reindex para garantizar que todos los meses aparezcan en orden calendario (ene-dic), incluso si no tienen datos.
```

```
    Args:
```

```
        df (DataFrame): DataFrame filtrado a un solo año (resultado de filtered_year)
```

```
    Returns:
```

```
        list[tuple[str, int]]: Lista de 12 tuplas (mes_abreviado, cantidad) en orden cronológico
```

```
    Examples:
```

```
        >>> df_full = load_dataframe("inscritos_test_sample.csv")
```

```
        >>> df_mini = reduce_cols(df_full)
```

```
        >>> df_2022 = filter_year(df_mini, 2022)
```

```
        >>> by_month(df_2022)
```

```
        [('ene', 11), ('feb', 21), ('mar', 13), ('abr', 16), ('may', 19), ('jun', 24), ('jul', 12), ('ago', 19), ('sep', 25),
```

```
        """]
```

```

month_order = [
    'ene', 'feb', 'mar', 'abr', 'may', 'jun',
    'jul', 'ago', 'sep', 'oct', 'nov', 'dic'
]

# Agrupamos el DataFrame por MES_INSCRIPCION y contamos
df_grouped = df.groupby('MES_INSCRIPCION')['FECHA_INSCRIPCION'].count()

# Reindexamos el resultado agrupado con el orden de meses.
df_ordered = df_grouped.reindex(month_order, fill_value=0)

# Convertimos la serie resultante a una lista de tuplas (mes, total)
result_list = list(df_ordered.items())

return result_list

```

In [26]: *# Casilla para hacer pruebas*

```

df_full = load_dataframe(TESTDATAFILE)
df_mini = reduce_cols(df_full)

# Definimos el año para la prueba. e.g. 2022
TEST_YEAR = 2022
df_2022 = filter_year(df_mini, TEST_YEAR)
print(f"DataFrame filtrado para el año {TEST_YEAR} listo. Filas: {len(df_2022)}")

EXPECTED_RESULT = [
    ('ene', 11), ('feb', 21), ('mar', 13), ('abr', 16),
    ('may', 19), ('jun', 24), ('jul', 12), ('ago', 19),
    ('sep', 25), ('oct', 19), ('nov', 20), ('dic', 18)
]

result = by_month(df_2022)

# Verificaciones
# a) Comprobar el número de meses
if len(result) == 12:
    print("El resultado contiene 12 meses.")
else:

```

```

    print(f"Error: Longitud incorrecta. Esperado: 12, Obtenido: {len(result)}.")

# b) Comprobar que el orden y los totales coinciden
if result == EXPECTED_RESULT:
    print("La lista de tuplas coincide con el resultado esperado y el orden es correcto.")
else:
    print("Error: Los resultados no coinciden o el orden es incorrecto.")
    print("Resultado Esperado (Primeros 3):", EXPECTED_RESULT[:3])
    print("Resultado Obtenido (Primeros 3):", result[:3])

# c) Mostrar el resultado completo
print("\nResultado completo de by_month:")
print(result)

```

DataFrame filtrado para el año 2022 listo. Filas: 217

El resultado contiene 12 meses.

La lista de tuplas coincide con el resultado esperado y el orden es correcto.

Resultado completo de by_month:

```
[('ene', 11), ('feb', 21), ('mar', 13), ('abr', 16), ('may', 19), ('jun', 24), ('jul', 12), ('ago', 19), ('sep', 25), ('oct', 19), ('nov', 20), ('dic', 18)]
```

Auto-comprobación

```
In [27]: bym = by_month(year_2022)
bym
```

```
Out[27]: [('ene', 804),
          ('feb', 2129),
          ('mar', 2257),
          ('abr', 1677),
          ('may', 2014),
          ('jun', 1727),
          ('jul', 1338),
          ('ago', 1511),
          ('sep', 1711),
          ('oct', 1939),
          ('nov', 2292),
          ('dic', 1800)]
```

Salida esperada

```
[('ene', 804),  
 ('feb', 2129),  
 ('mar', 2257),  
 ('abr', 1677),  
 ('may', 2014),  
 ('jun', 1727),  
 ('jul', 1338),  
 ('ago', 1511),  
 ('sep', 1711),  
 ('oct', 1939),  
 ('nov', 2292),  
 ('dic', 1800)]
```

Comprobaciones automáticas

```
In [28]: testing.test_by_month(by_month, filter_year, load_dataframe)
```

OK

```
In [29]: testing.test_type_hints_by_month(by_month)
```

by_month type hints OK

```
In [30]: testing.test_docstring(by_month)
```

docstring OK

```
In [31]: testing.test_doctests(by_month, globals())
```

doctests OK

Gráficas

Finalmente, vamos a realizar un par de gráficos. En primer lugar, queremos sacar el número de personas inscritas por cada mes. Realiza una función `show_by_month` que reciba como dato de entrada un fichero de datos csv y un año y que realice gráfico de barras del número de desempleados por meses de ese año

```
In [32]: def show_by_month(datafile: str, year: int) -> None:
        """
        Genera un gráfico de barras mostrando inscripciones mensuales para un año específico.

        Ejecuta el pipeline completo: carga --> reduce --> filtra --> agrupa --> visualiza.
        Muestra los 12 meses en orden cronológico con el conteo de inscripciones.

        Args:
            datafile (str): Ruta del archivo csv de inscripciones
            year (int): Año a visualizar (e.g. 2019)
        """

        df_full = load_dataframe(datafile)

        # Reducimos columnas
        df_reduced = reduce_cols(df_full)

        # Filtramos por el año deseado
        df_year = filter_year(df_reduced, year)

        # Obtenemos los totales mensuales en el formato [(mes, total)]
        monthly_data = by_month(df_year)

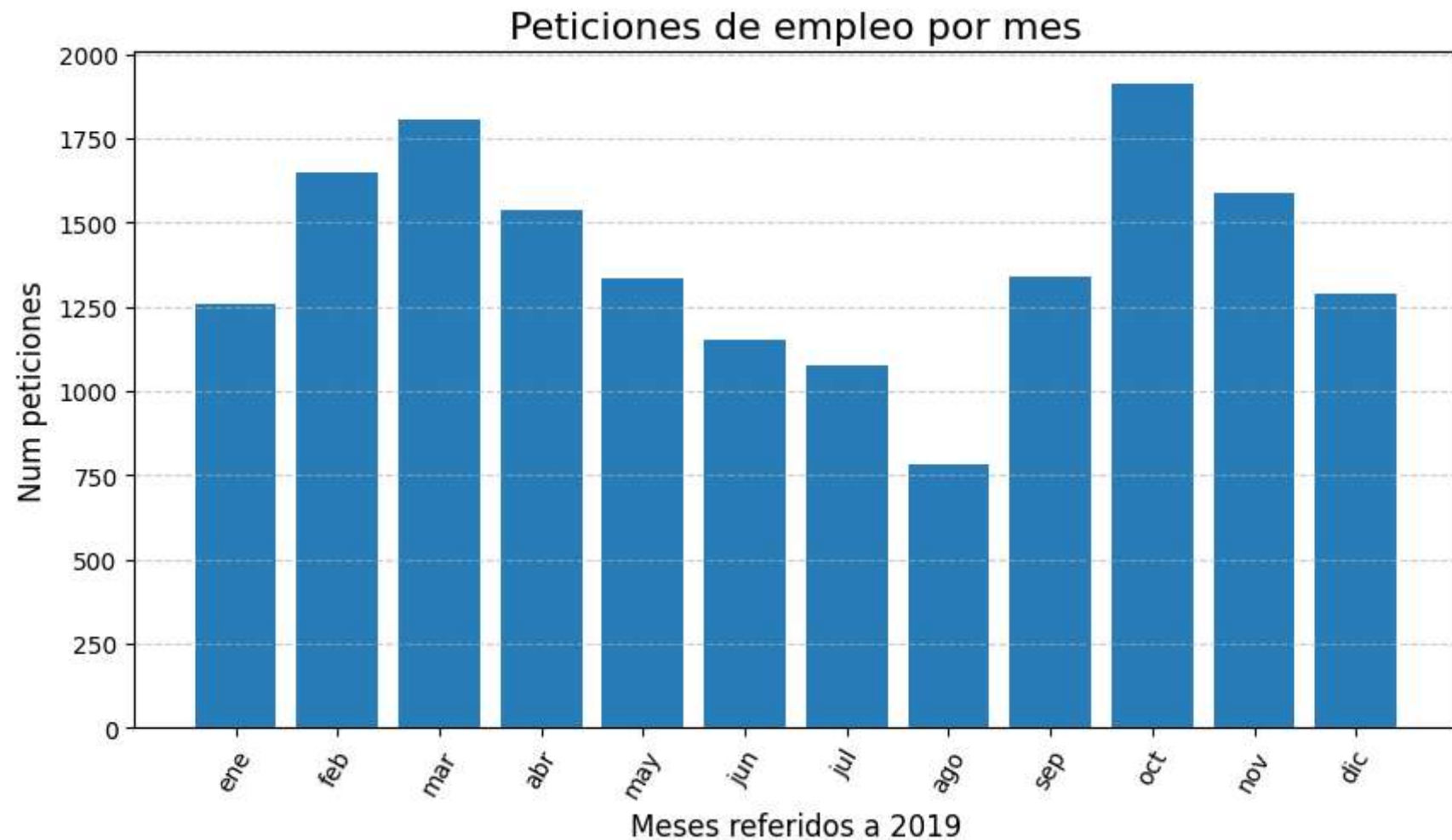
        # Preparación para La Gráfica
        # Descomponemos la lista de tuplas en dos listas separadas (Meses y Totales)
        months, counts = zip(*monthly_data)

        # Generación de La Gráfica de Barras
        plt.figure(figsize=(10, 6))
        plt.bar(months, counts, color='#2980b9')
        plt.title(f'Peticiones de empleo por mes', fontsize=16)
        plt.xlabel(f'Meses referidos a {year}', fontsize=12)
        plt.ylabel('Num peticiones', fontsize=12)
```

```
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.xticks(rotation=60) # Rota el eje 'x'
plt.subplots_adjust(bottom=0.22)

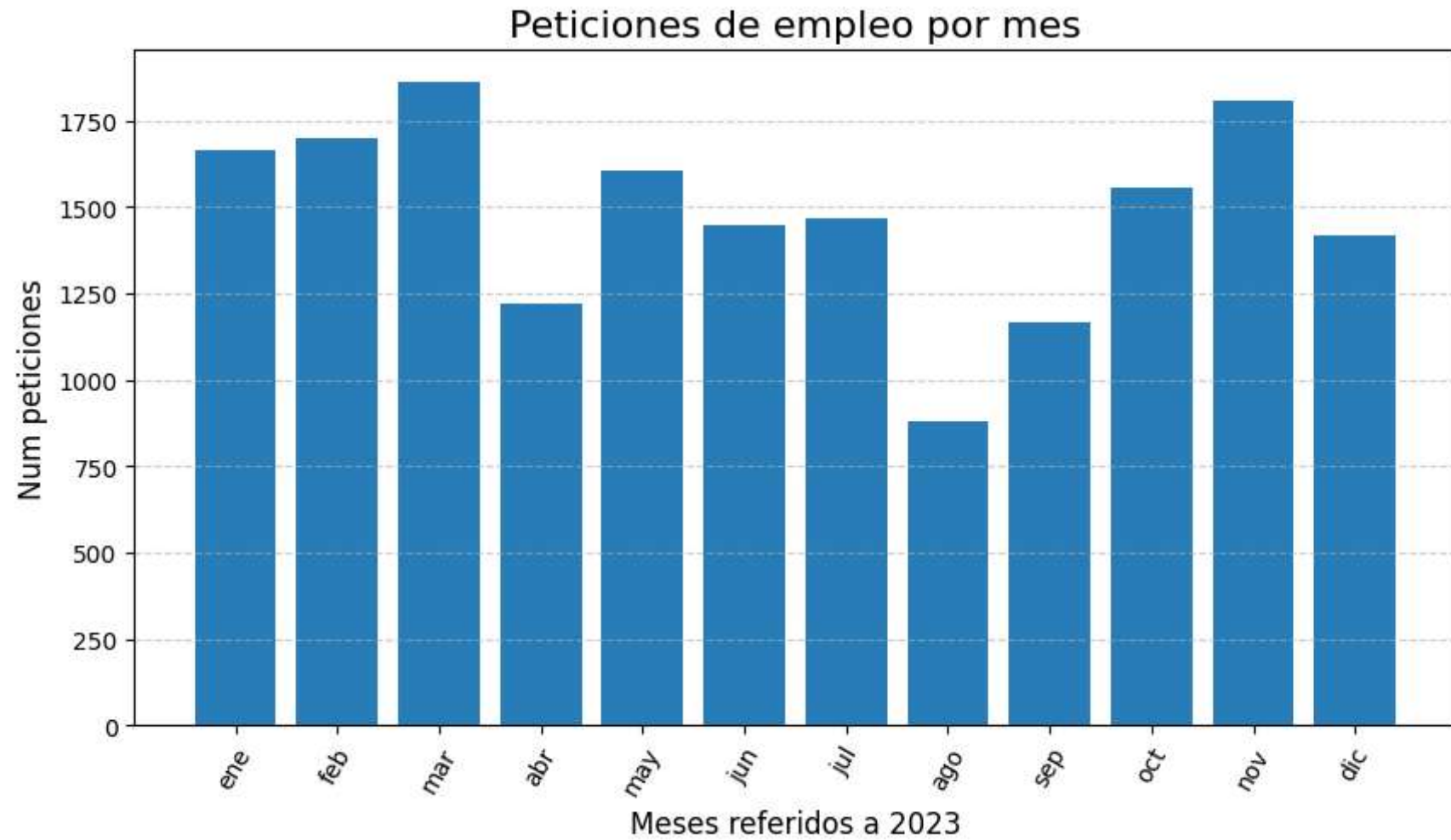
plt.show()
```

```
In [33]: # Casilla para pruebas
# Comprobamos que la gráfica funciona para cualquier año
show_by_month('inscritos.csv', 2019)
```

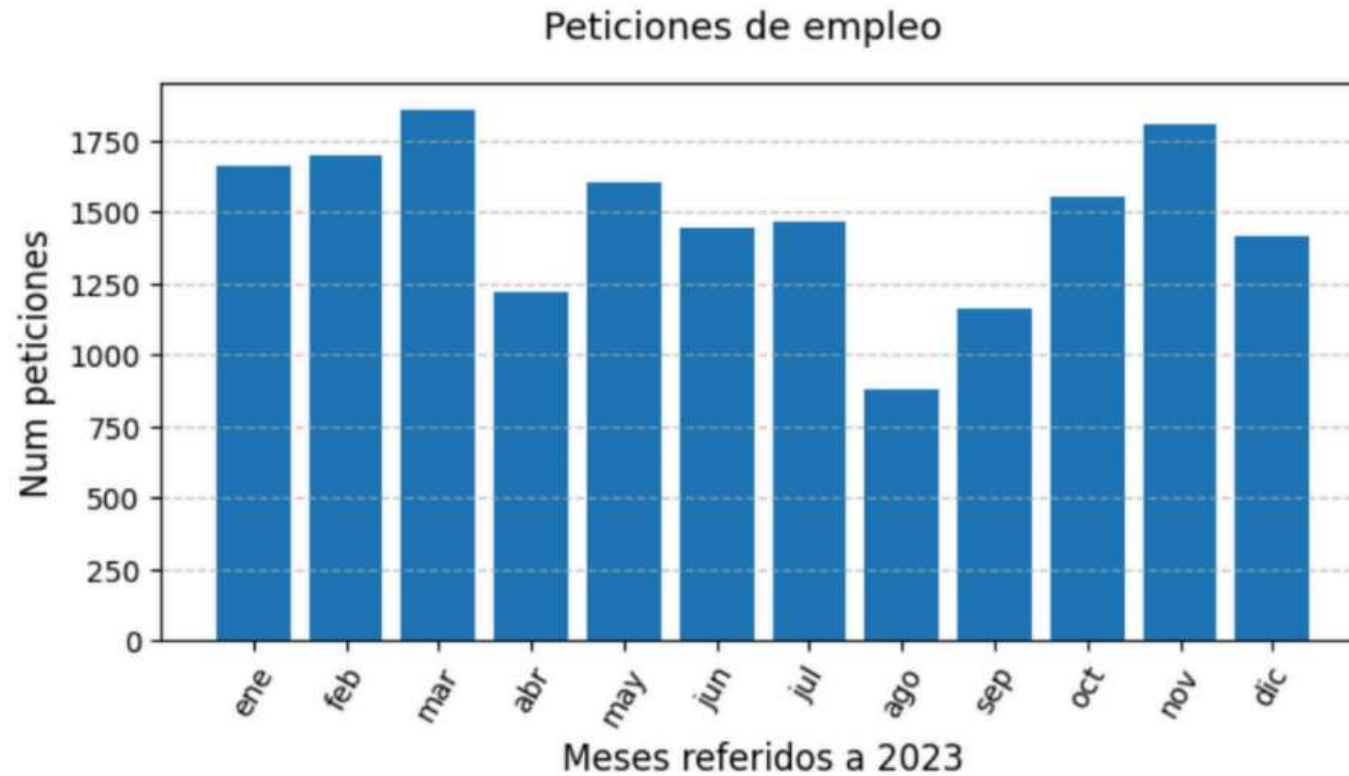


Auto-comprobación

```
In [34]: show_by_month('inscritos.csv', 2023)
```



Salida esperada



Inscritos por distritos en un año

Realiza una función `show_district` que genere un gráfico de tarta del número de desempleados por distrito en un determinado año. Al igual que en el caso anterior, la función recibe como parámetros el nombre de un fichero con los datos y el año. Si lo haces verás que el distrito que más se repite es 'OTRO MUNICIPIO'. Debes filtrar los datos, de forma que solo se consideren los distritos de Madrid.

```
In [35]: # Asumimos que load_dataframe, reduce_cols, y filter_year están definidas

def show_district(datafile: str, year: int) -> None:
    """
    Genera un gráfico de tarta con el número de inscritos por distrito de Madrid.

    Ejecuta el pipeline completo y agrupa por distritos. Excluye 'OTRO MUNICIPIO'.
```

El distrito con más inscripciones se resalta (explode) para destacarlo visualmente.
Los distritos se ordenan de mayor a menor número de inscripciones.

Args:

```
datafile (str): Ruta del archivo csv de inscripciones
year (int): Año a visualizar (e.g. 2020)
"""

# Pipeline de Procesamiento (Cargar -> Reducir -> Filtrar por Año)
df_full = load_dataframe(datafile)
df_reduced = reduce_cols(df_full)
df_year = filter_year(df_reduced, year)

# Filtramos Distritos: Exclusión de 'OTRO MUNICIPIO'
df_madrid = df_year[df_year['DISTRITO_DESC'] != 'OTRO MUNICIPIO']

# Agrupamos, contamos y ordenamos por Distrito (Mayor a Menor)
district_counts = (
    df_madrid.groupby('DISTRITO_DESC')['FECHA_INSCRIPCION']
    .count()
    .sort_values(ascending=False)
)

# Como el DataFrame ya está ordenado de mayor a menor, el máximo siempre es el índice 0.
num_slices = len(district_counts)

# Creamos una lista 'resalto' con ceros (sin separación) para todas las porciones
# Será la lista que utilizemos para resaltar el distrito con mayor nº de peticiones de empleo
explode = [0] * num_slices

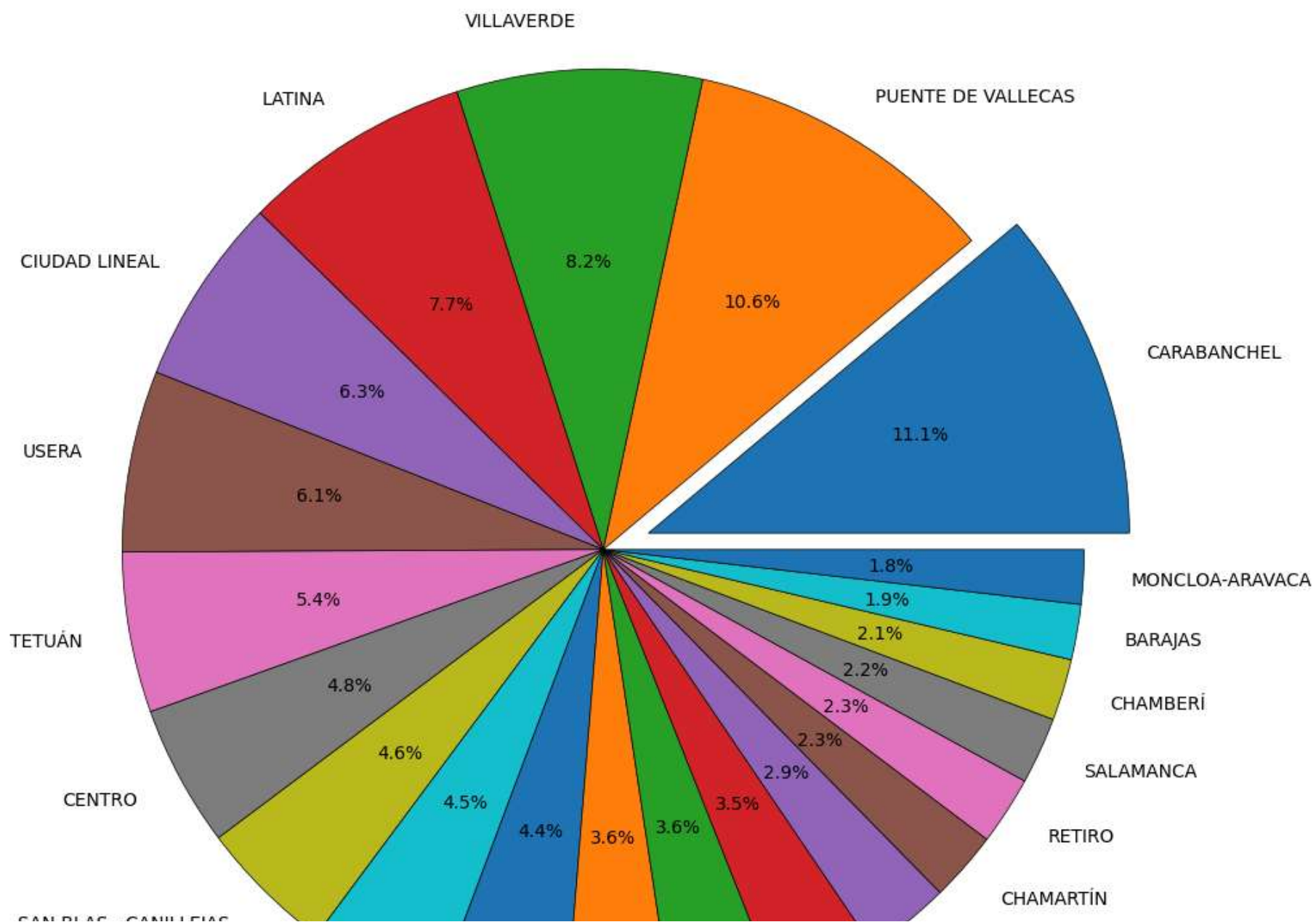
# El máximo siempre está en el índice 0. Asignamos la separación (0.1) a esa posición.
explode[0] = 0.1

# Generamos el Gráfico de Tarta
plt.figure(figsize=(12, 12))
district_counts.plot.pie(
    autopct='%1.1f%%',
    startangle=0,
    wedgeprops={'linewidth': 0.5, 'edgecolor': 'black'},
    textprops={'fontsize': 10},
    explode=explode
```

```
)  
plt.title(f'Peticiones de empleo por distrito', fontsize=16)  
plt.ylabel('')  
plt.text(x=0.5, y=-0.05,  
         s=f'Datos referidos a {year}', # Los datos del año que quieres mostrar  
         ha='center', # Alineación horizontal: centrado  
         transform=plt.gca().transAxes, # Usa Las coordenadas del eje actual  
         fontsize=12)  
  
plt.show()
```

```
In [36]: # Casilla para pruebas  
# Comprobamos que la gráfica funciona para cualquier año  
show_district('inscritos.csv', 2020)
```

Peticiones de empleo por distrito

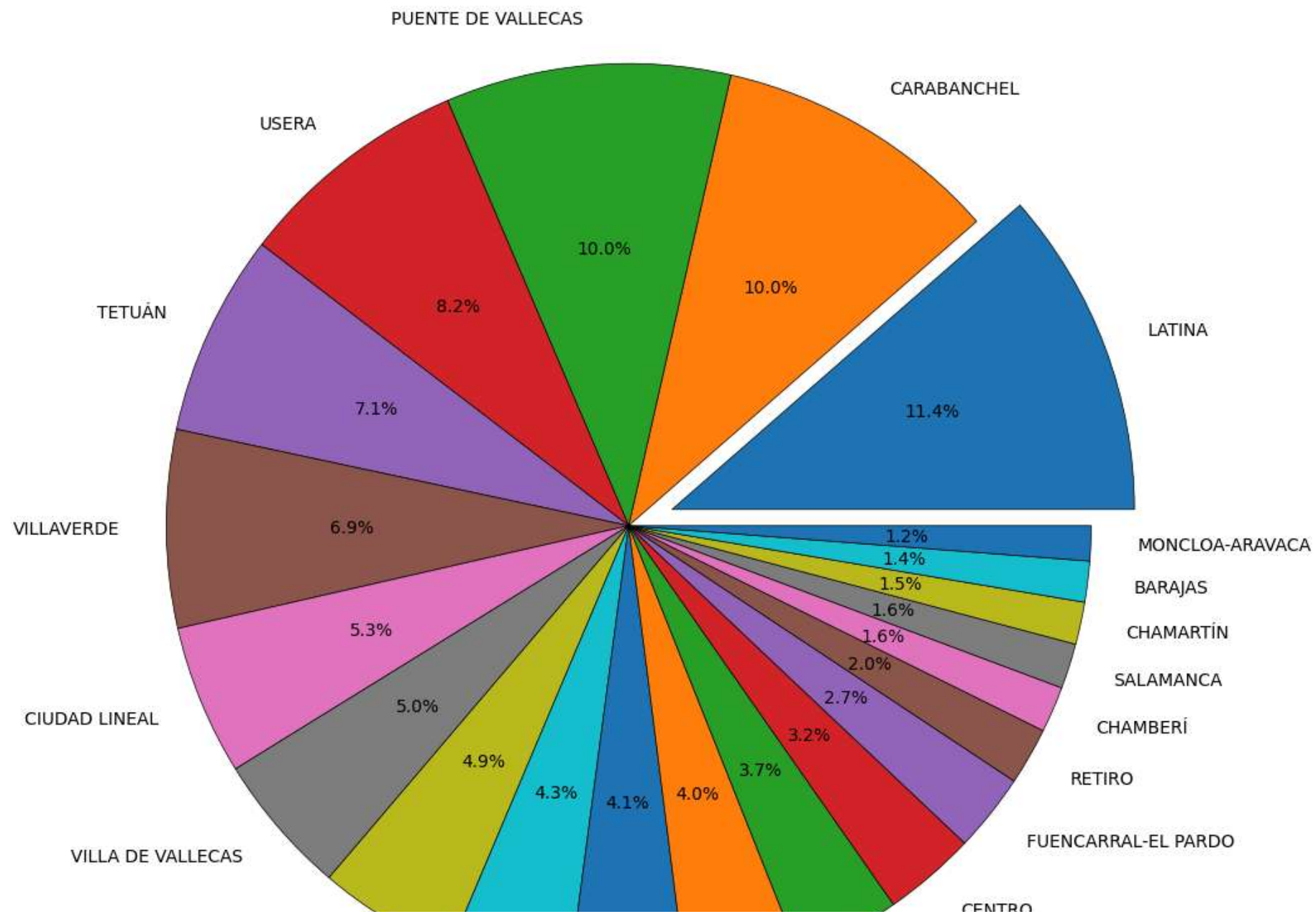




Datos referidos a 2020

```
In [37]: show_district('inscritos.csv', 2022)
```

Peticiones de empleo por distrito





Datos referidos a 2022

Salida esperada

